

TASK 2 — Data-Driven Automation in Python Behave

1. Experiment Title

Data-Driven Automation Using Python and Behave

2. Problem Statement

Implement data-driven login automation using **Python and Behave BDD**, where the same login scenario is executed with multiple sets of usernames, passwords, and expected results.

3. Objective

- To implement data-driven testing using Behave.
- To use **Scenario Outline** and **Examples** in Gherkin.
- To execute the same test scenario with different test data.
- To verify both successful and unsuccessful login attempts.

4. Tools, Software and Concepts Used

Tools & Software

- Python
- Behave
- Selenium WebDriver
- Google Chrome
- PyCharm / VS Code

Concepts Used

- Behaviour-Driven Development (BDD)
- Gherkin
- Scenario Outline
- Examples Table
- Data-Driven Testing
- Selenium WebDriver

- Web Element Locators
- Assertions

5. Source Code / Implementation Steps

Project Structure

Assignment_2_Behave_DDT

```
|
└─ features
    |
    └─ login.feature
        |
        └─ steps
            └─ login_steps.py
```

login.feature

Feature: Data Driven Login

Scenario Outline: Login with different credentials

Given I open the SauceDemo login page

When I enter username "<username>" and password "<password>"

And I click the login button

Then the login result should be "<result>"

Examples:

username	password	result	
standard_user	secret_sauce	success	
wrong_user	wrong_pass	error	

| standard_user | wrong_pass | error |

login_steps.py

```
from behave import given, when, then
```

```
from selenium import webdriver
```

```
from selenium.webdriver.common.by import By
```

```
@given("I open the SauceDemo login page")
```

```
def step_open_login_page(context):
```

```
    context.driver = webdriver.Chrome()
```

```
    context.driver.maximize_window()
```

```
    context.driver.get("https://www.saucedemo.com/")
```

```
@when("I enter username \"{username}\" and password \"{password}\"")
```

```
def step_enter_credentials(context, username, password):
```

```
    context.driver.find_element(By.ID, "user-name").send_keys(username)
```

```
    context.driver.find_element(By.NAME, "password").send_keys(password)
```

```
@when("I click the login button")
```

```
def step_click_login(context):
```

```
    context.driver.find_element(
```

```
        By.XPATH, "//input[@type='submit']"
```

```
    ).click()
```

```
@then('the login result should be "{result}"')
def step_verify_login_result(context, result):

    if result == "success":

        assert "/inventory.html" in context.driver.current_url

        print("Login successful")

    elif result == "error":

        error_message = context.driver.find_element(

            By.CSS_SELECTOR, "h3[data-test='error']"

        ).text

        assert "Username and password do not match" in error_message

        print("Login failed as expected")

    context.driver.quit()
```

Execution

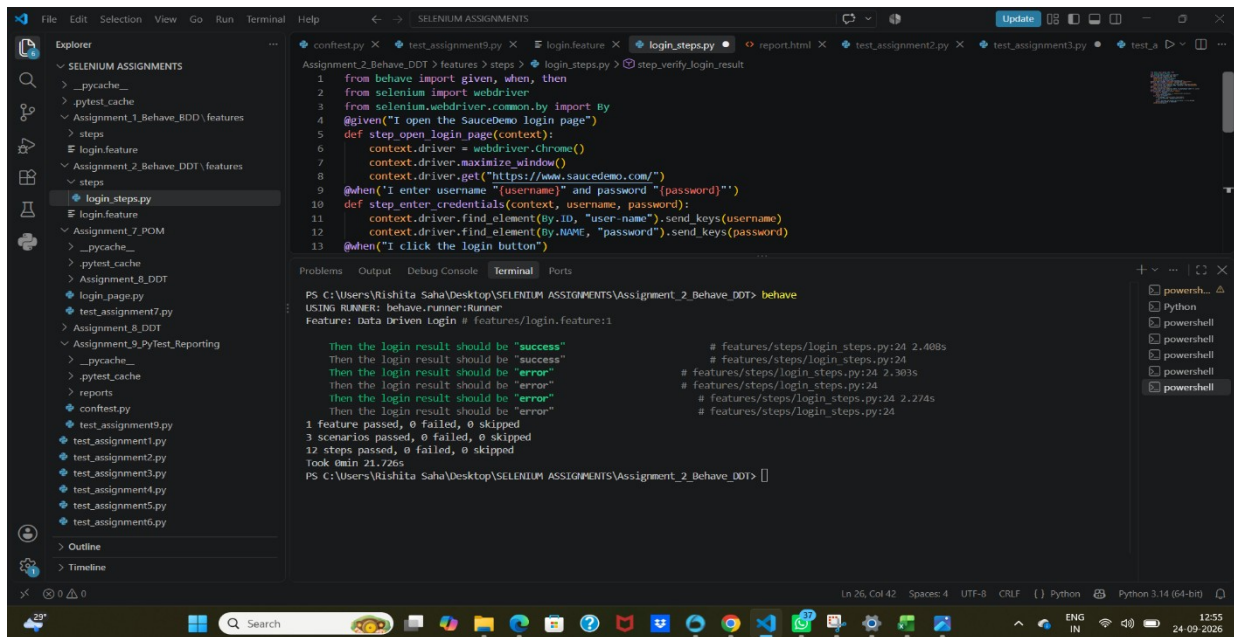
Navigate to the Assignment 2 folder and run:

behave

The three rows in the Examples table cause the scenario to execute three times:

1. Valid username + valid password → **success**
2. Invalid username + invalid password → **error**
3. Valid username + invalid password → **error**

6. Output with Clearly Labelled Screenshots



Behave Execution Result

Show the terminal after:

behave

The execution should show all three scenarios passing.

7. Result

The login functionality was successfully automated using **data-driven testing in Behave**. The same Scenario Outline was executed with multiple sets of credentials, covering both successful and unsuccessful login cases.

8. Observation

- Scenario Outline allows the same scenario to be executed with different data.
- The Examples table provides the test data.
- Three different credential combinations were tested.
- Successful and unsuccessful login results were verified using assertions.

9. Conclusion

This experiment demonstrated **data-driven automation using Python and Behave BDD**. The Scenario Outline and Examples table enabled multiple login combinations to be tested using a single reusable test scenario.